

From Requirements to Agentic Development.

02 — FROM REQUIREMENTS TO AGENTS

60 MIN

How we evolve from a user need to a working spec — and what the coding agent does with it.

Lecture Series Roadmap

L01 Software Engineering in the Age of AI: Core Concepts



L02 **From Requirements to Agentic Development**

L03 System Design, Architecture & Agent Orchestration

UPCOMING

L04 Deploying, monitoring, and Testing Agents

UPCOMING

L05 Security & Ethics

UPCOMING

PART 01

Lecture 1

Recap.

Where we've been — and where today takes us.

What we covered last time.

- | | | |
|----|--|--|
| 01 | Engineering vs. programming. | Requirements, design, validation, change — code is one ingredient. |
| 02 | Deterministic vs. probabilistic systems. | Same input, different output. Behavior depends on context and phrasing. |
| 03 | Foundations of language models. | Tokens, transformers, attention, hallucination, bias, cost. |
| 04 | Practical implications. | Specification, validation, edge cases, oversight — the engineer's new job. |
| 05 | The AI-driven SDLC. | How the workflow shifts — prompt → gen → validate → refine. |

Where we're going today.

One running example. Three levels of precision. Then the tool that consumes it.



Discussion Think about the last app you used. What do you think the original requirement was for the login screen — and how was it communicated to the engineers?

The #1 cause of software project failure.

Getting requirements wrong in the first place.

- Requirements that describe HOW instead of WHAT.
- Requirements so vague they're just guesses — no one can build from them.
- No requirements at all — just high-level descriptions of a solution.

IEEE DEFINITION

"A condition or capability needed by a user to solve a problem or achieve an objective."

Note: leaves no room for describing HOW — only WHAT.

What is a user story?

WHO WRITES IT: Product Manager · LANGUAGE: User language · ANSWERS: Why & what the user needs

"As a new user, I want to create an account so that I can access my personalized dashboard."

"As a returning user, I want to log in securely so that I can pick up where I left off."

✓ Outcome-focused ✓ Technology-agnostic ✗ Not precise enough for engineers to build from

User stories are short, simple, and informal descriptions, written from the perspective of the end user or customer. They focus on user needs—what they want to achieve and why—rather than technical specifications

What is *requirements management*?

THE STRATEGIC LEVER: Documentation, Traceability, Analysis, and Prioritization

DEFINITION

The systematic process of **gathering, analyzing, and validating** the needs of a system to ensure it meets stakeholder expectations.

It acts as a **detailed roadmap** from inception to commercialization.

WHY IT MATTERS

- **Risk Reduction:** Identifies ambiguities before they become design conflicts.
- **Cost Control:** Early error detection is up to 1,500x cheaper than fixing issues in operations.
- **Alignment:** Anchors teams to a "Single Source of Truth."

*"Requirements management ensures the product is **built correctly**, whereas project management ensures it's built efficiently."*

Example requirements

WHO WRITES IT: PM + Engineer · LANGUAGE: Business/system · ANSWERS: What the system must do

- The system must allow users to register with an email and password.
- Passwords must meet minimum security standards.
- The system must prevent duplicate accounts for the same email.
- The system must reject invalid or unrecognized credentials.
- The system must support password recovery.
- The system must lock accounts after repeated failed login attempts.

Still technology-agnostic. An engineer knows WHAT to build — but not yet HOW.

Types: Functional vs. non-functional requirements.

FUNCTIONAL

What the system DOES.

- Register with email & password
- Authenticate returning users
- Reject invalid credentials
- Lock accounts after n failures
- Support password recovery

NON-FUNCTIONAL

How WELL it does it.

- Security: passwords meet cryptographic standards
- Performance: auth response < 300ms
- Rate limiting: 10 req/min per IP
- Availability: 99.9% uptime
- Scalability: 10k concurrent logins

Both must be captured before writing a spec. Non-functional requirements are where most teams cut corners.

Common Mistakes

- 01. Writing Vague or Ambiguous Requirements
- 02. Combining Multiple Requirements into One
- 03. Mixing Functional and Non-Functional Requirements
- 04. Ignoring Testability
- 05. Overloading with Technical Jargon
- 06. Failing to Include Rationale
- 07. Not Regularly Reviewing Requirements
- 08. Over-Engineering the Requirements

Common Mistakes

✗ BAD REQS

- The system should process requests quickly
- The system shall validate user credentials, display a welcome message, and send a verification email.
- The system shall process up to 1,000 transactions per second and ensure user data security
- The system shall provide an exceptional user experience

✓ GOOD REQS

- Response within 0.3 seconds
- Break it into multiple requirements
- Separate these into functional and non functional reqs requirements
- The system shall achieve a score of 90 or higher on standardized usability testing.

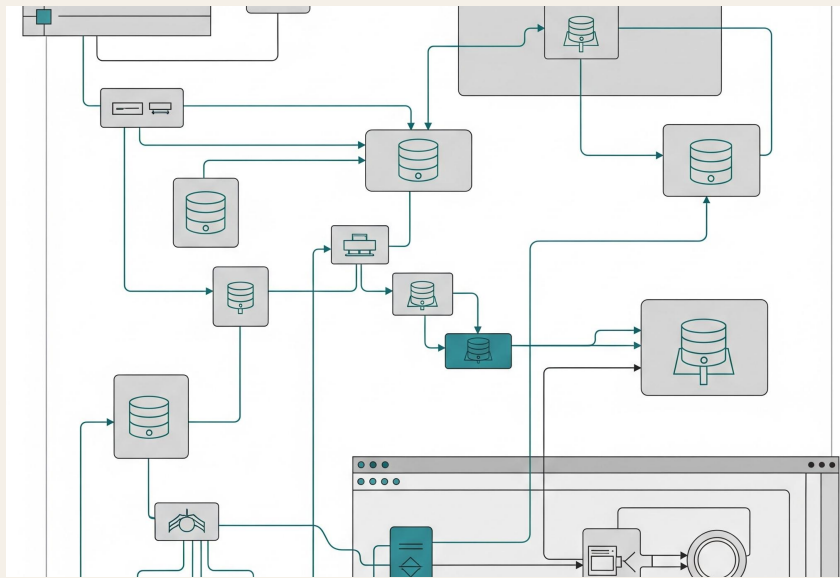
The technical specification

WHO WRITES IT: Engineer / Tech Lead • LANGUAGE: Implementation-specific • ANSWERS: HOW the system will be built

THE BLUEPRINT

A **comprehensive document** outlining internal programming, data schemas, and technical standards.

It acts as a **contract** between architects and developers to ensure scalability and security.



*"The tech spec bridges the gap between **business requirements** and **technical implementation**, reducing risk before the first line of code."*

Writing a good spec.

Anatomy, anti-patterns, and the punchline.

Anatomy of a technical spec.

Header	Title · Author · Reviewers · Date · Ticket reference
Introduction	Problem summary · Glossary · Context · Goals · Assumptions
Solution	Current state · Proposed design · Data model · API changes · Error states · Edge cases
Test Plan	Unit tests · Integration tests · QA criteria · How tests verify requirements
Rollout Plan	Deployment phases · Monitoring · Alerting · Roll-back procedure

the spec.

WHO WRITES IT: Engineer · LANGUAGE: Technical · ANSWERS: What & how at implementation level

POST /auth/register

accepts email + password · returns 201 or 409 if duplicate

Password rules

min 8 chars, 1 uppercase, 1 number, 1 special char · validation client + server side

POST /auth/login

returns signed JWT (24hr) + refresh token (30 days)

Account logout

counter increments per failure · locked after 5 attempts · unlocks after 15 min or via email

POST /auth/reset-password

signed link via SendGrid · expires 15 min

Rate limiting

10 requests/min per IP on all auth endpoints

Password storage

bcrypt hash · salt rounds = 12

At a glance: User Stories vs Requirements vs Specs.

	User Story	Requirements	Spec
Written by	PM	PM + Engineer	Engineer
Language	User language	Business / system	Technical
Answers	Why & what (user)	What (system must do)	What & how (engineering)
Engineer/Agent can use it?	Too vague ✗	Partially ◐	✓ This is agent fuel

Discussion How much specifications before implementation do we need?

Bad spec vs. good spec.

✗ BAD SPEC

Describes HOW not WHAT.

Focuses on UI: "user clicks the blue login button"
Mixes what and how inconsistently
Missing error states — 6th failed login?
No non-functional requirements at all
Written once, never updated again

✓ GOOD SPEC

A contract for builders.

Technology-agnostic where possible, precise where it matters
Edge case named, not just the happy path
Non-functional requirements explicit and measurable
Written to be consumed by a human or a coding agent
Lives alongside the code and evolves with it

Discussion If you handed the login spec to Claude Code right now, what would it get right? What would it probably miss?

Engineers/Agents consume specs. not user stories.

A user story is too vague to drive a machine that moves at the speed of code generation.

A well-written spec is the contract between human intent and machine execution.

LLMs vs. reasoning models vs. agents.

LLM

THE ENGINE

Next-token prediction model.
Estimates the probability of the next token in a sequence.
The base layer everything else builds on.

Reasoning Model

BEEFED-UP ENGINE

Still an LLM. Trained to solve multi step problems.
More powerful, more expensive to run.

Agent

THE CONTROL LOOP

A harness around the model.
Agents observe, inspect, choose, act.
Decides what to inspect next, which tools to call, when to stop.

The LLM is the engine. A reasoning model is a beefed-up engine. An agent harness makes the model useful for real work.

What is an agent?

the system that repeatedly calls the model inside an environment.



Brain (Model)

The reasoning core. An LLM that handles logic, planning, and language.



Memory

Working: Active task context.
Long-term: Persistent knowledge.



Tools & Actions

APIs and interpreters to execute tasks in the real world.



Planning

Decomposing complex goals (e.g., ReAct, Plan-and-Execute).



Perception

Ingesting text, voice, or sensor data from the environment.



Reflection

A self-correction loop to review outcomes and fix errors.



Guardrails & Policy: Security frameworks ensuring safe and compliant operations.

An agent is more than a model; it is a system of reasoning, storage, and action. Unlike a simple LLM, it has the "agency" to interact with tools and evaluate its own work until a goal is met.

What is a coding agent?

Agent that codes

An agent that plans, writes, tests, and debugs code by iterating on high-level instructions.

Key Capabilities

- create files, write code, run tests, and fix errors
- They understand existing codebases
- Environment interaction
- Agents can use terminal tools, CLIs and browser automation

Environment & Tools: Unlike basic autocomplete, agents use terminal commands, browser automation, and file management to complete multi-step tasks independently.

Popular Examples

Cursor
Codex
Claude Code

PART 04

Components of a coding agent.

The tool that will consume the spec you just wrote.

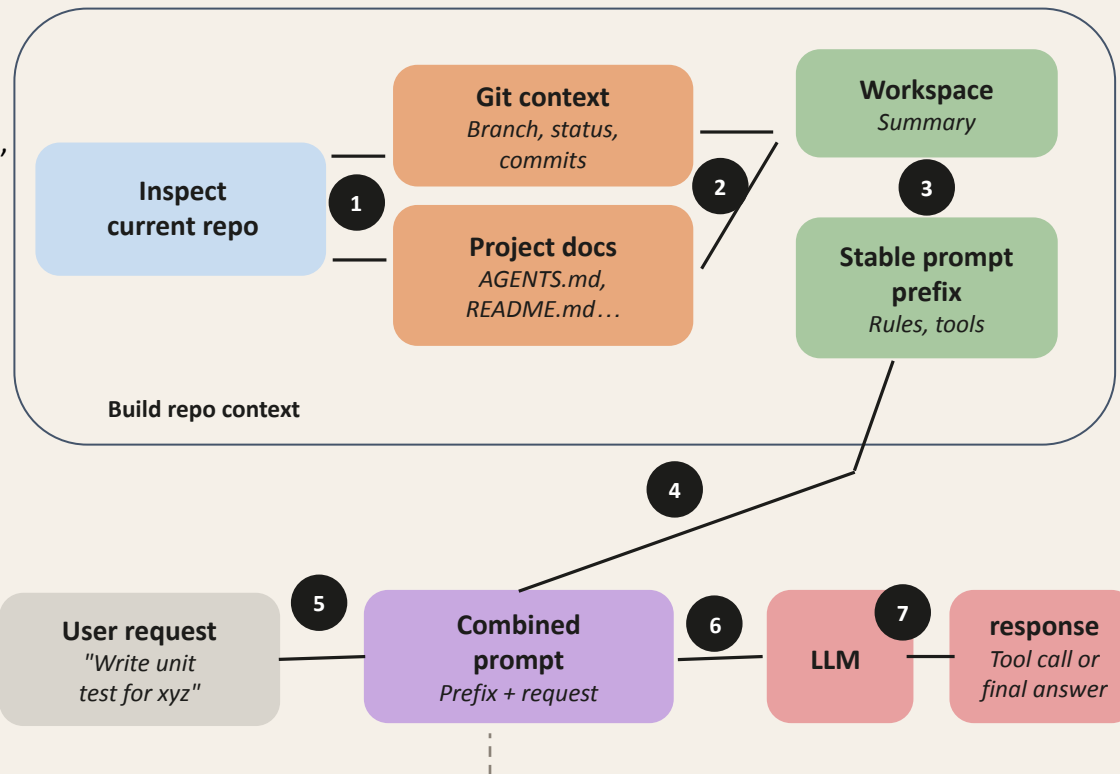
Components of a coding agent.

01	Live repo context	Git branch, status, commits, project docs — collected upfront before any work begins.
02	Prompt shape & cache reuse	Stable rules and workspace summary are cached. Only the request and memory change each turn.
03	Structured tools, validation & permissions	File edit, terminal, search — with validation and human approval gates on sensitive actions.
04	Context reduction & output management	Clipping, deduplication, and asymmetric detail keep the prompt window from overflowing.
05	Delegation & bounded subagents	Main agent spawns bounded child agents for parallel subtasks — inheriting context, constrained by scope.

Live repo *context*.

When a user says “fix the tests” or “implement xyz,” the model must know the repo layout, branch, and project docs before acting.

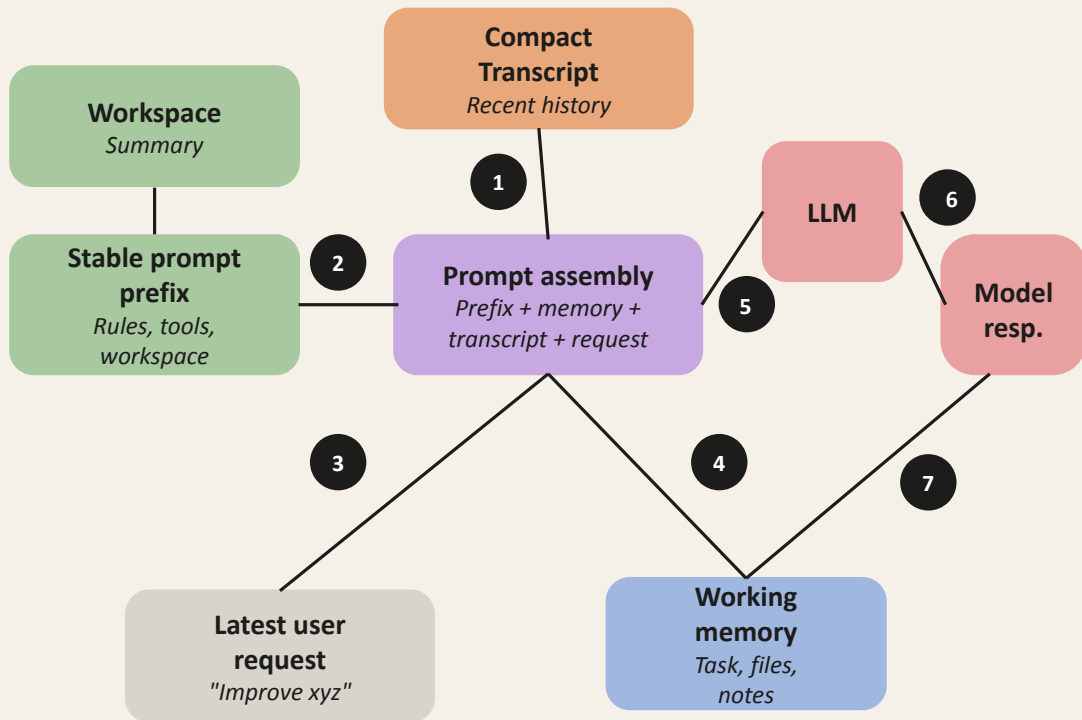
- *Stable facts*. collected upfront, not re-fetched every prompt.
- *Git context*. branch, status, and recent commits.
- *Project docs*. AGENTS.md, README — instruction files the model reads first.
- *Workspace summary*. combined into a stable prompt prefix reused across turns.



Prompt shape & *cache reuse*.

Coding sessions are repetitive. Agent rules, tool descriptions, and workspace summaries stay mostly the same. Only the request, transcript, and short-term memory change each turn.

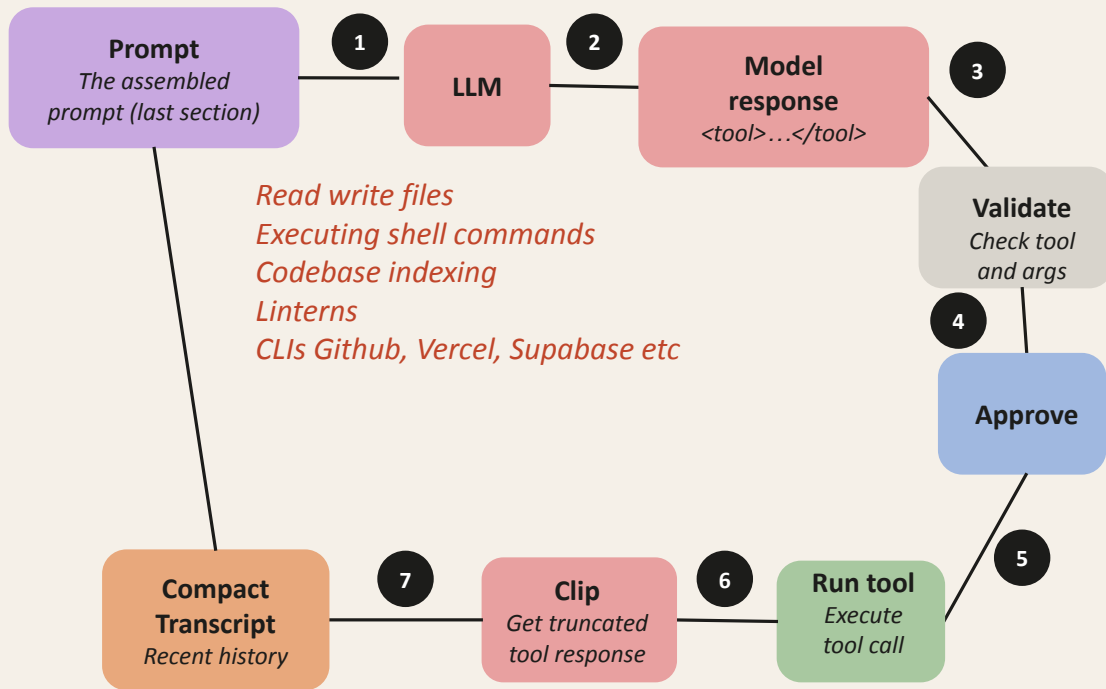
- **Stable prefix.** rules + tools + workspace summary — cached, not reprocessed.
- **Compact transcript.** recent history, compressed — for prompt reconstruction.
- **Working memory.** current task, files, notes for task continuity.
- **Cache reuse.** avoids re-processing stable context on every query.



Tools access and use

Tools are what separate an agent from a chat box. But raw tool calls need validation and human approval before executing.

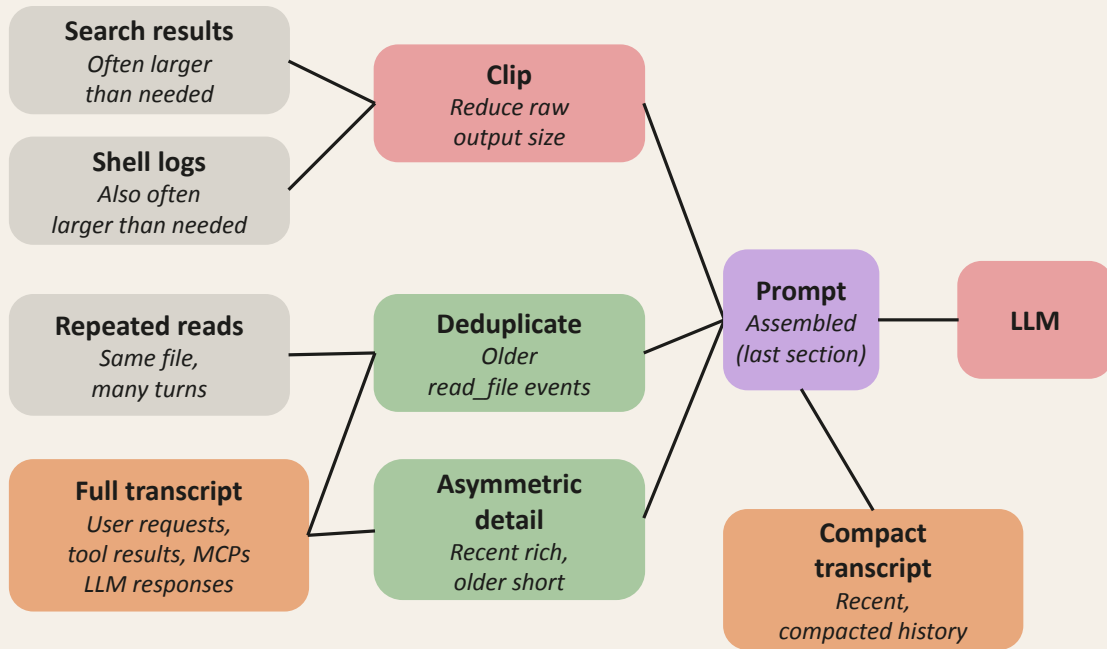
- **Validate.** check tool name and arguments before executing.
- **Approve.** human yes/no gate on sensitive or destructive actions.
- **Run tool.** execute file edit, terminal command, or search.
- **Clip.** truncate large tool outputs to only the important info.
- **Loop.** clipped result fed back into compact transcript → next prompt.



Minimizing Context Bloat

Context windows are finite. Long sessions accumulate fast. The agent must actively manage what survives into the next prompt.

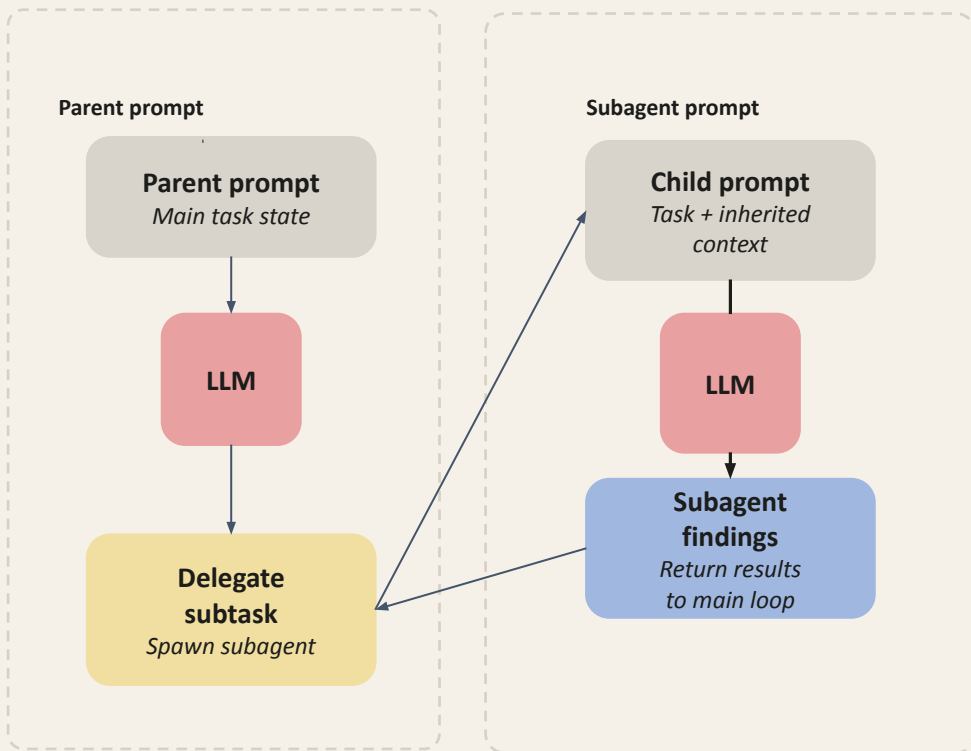
- **Clip**. reduce raw output size from search results and shell logs.
- **Deduplicate**. collapse repeated reads of the same file.
- **Asymmetric detail**. recent turns stay rich; older turns get summarised.
- **Compact transcript**. assembled from the above; fed into each new prompt.



Delegation with *subagents*.

Once an agent has tools and state, delegation allows it to parallelize subtasks without forcing one loop to carry every thread of work at once.

- *Inherit context.* subagent gets enough to be useful.
- *Return findings.* result returned to main loop
 - not full transcript.
- *Parallel.* main agent continues while child works side task.



Test Runner, Security Reviewer,
Performance Reviewer

PART 06

Working with agents.

Understanding, building, and knowing when to start over.

PART 05

Spec-driven development.

The brakes on agentic coding.

The spec-driven workflow in practice.

1 Write the spec before opening the agent.

Define requirements, edge cases, and non-functionals first.

2 Feed spec as the initial prompt.

Not a vague description — the full spec as input context.

3 Agent generates a step-by-step plan.

Asks clarifying questions, breaks work into verifiable steps.

4 Review and edit the plan.

You edit the plan before execution begins. Catch wrong directions early.

5 Agent builds incrementally and verifies.

Tests are the checkpoints. Agent runs, fails, adjusts, retries.

6 Update the spec as you learn.

Keep it alive as the system evolves. Spec-anchored, not spec-once.

The plan → TDD → verify loop.

1 Start with a plan.

Agent researches codebase, asks questions, produces an editable step-by-step plan.

2 Write tests first.

Describe expected inputs, outputs, error cases. Do NOT write the function yet.

3 Confirm tests fail.

Tell the agent to run tests and verify they fail. You're locking in requirements.

4 Commit the tests.

Lock in requirements before any feature code exists. Agent cannot modify them.

5 Agent writes the code.

Make all tests pass without modifying tests. Iterate until everything passes.

6 Verify and commit.

Review the output, confirm it behaves as expected, ship.

When to start over vs. iterate.

KEEP ITERATING WHEN...

The direction is right.

Agent is mostly correct but needs refinement
Architecture and direction are sound
Small misunderstandings follow-up can fix
You're mid-feature with useful context

START OVER WHEN...

The foundation is wrong.

Agent built the wrong architecture entirely
Missed a key system design constraint upfront
Agent is going in circles — start a fresh conversation
More than 3 follow-up corrections have not fixed it

Starting over from a better plan is almost always faster than patching a wrong direction.

What to leave with.



The substrate changed. You are now directing a machine that moves at the speed of code generation.

The discipline didn't. Clarity of intent, testable behavior, incremental verification — these still matter more, not less.

Speed amplifies everything. Good specs and clear requirements become more valuable, not less, when an agent can act on them instantly.

If a future employer asked: "how do you make sure an AI coding agent builds what you actually wanted?" — what would you say after today?

Lecture Series Roadmap

L01

Software Engineering in the Age of AI



L02

From Requirements to Agentic Development



L03

System Design, Architecture & Agent Orchestration



L04

Testing Agents

UPCOMING

L05

Security & Ethics

UPCOMING